

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 03 -

CONTROLE DE VERSÃO PARA ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS.....	17

EMSE

O QUE VEM POR AÍ?

Nesta aula, vamos aprender sobre a ferramenta de gerenciamento de versões mais utilizada por pessoas desenvolvedoras, o git.

EMEND

HANDS ON

Nesta aula vamos aprender os conceitos e comandos básicos de git. Vamos entender porque não é recomendado colocar dados em repositórios git, e como podemos criar e juntar fluxos de trabalho paralelos.



SAIBA MAIS

Como vimos em aulas passadas, um dos princípios de MLOps é o versionamento. Mas o que é isso? Talvez você não saiba, mas provavelmente esse processo já foi realizado por você no seu dia a dia. Vamos discutir a partir de um exemplo, o que significa versionar, e depois vamos especificar qual é a forma de se versionar códigos e dados mais utilizada hoje em dia.

Suponha que você está desenvolvendo um projeto. Você escreve, inicialmente, o conteúdo em um arquivo chamado projeto_v0, e ao longo do tempo você precisa fazer algumas alterações, mas gostaria de manter o conteúdo do arquivo inicial para referência no futuro. Existem algumas opções, uma delas é nunca apagar nenhum conteúdo do arquivo inicial, isto é, você mantém o texto original e suas alterações no mesmo arquivo, conforme mostra a Figura 1.

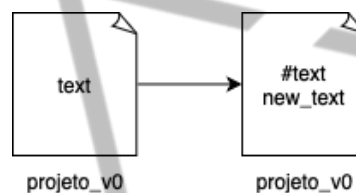


Figura 1 - Controle de versão utilizando um único arquivo
Fonte: Elaborado pelo autor (2024)

Essa forma de “versionar” pode ficar insustentável rapidamente. Talvez, uma melhor opção seja criar um arquivo para cada alteração, o que provavelmente você já fez ou faz, conforme mostra a Figura 2:

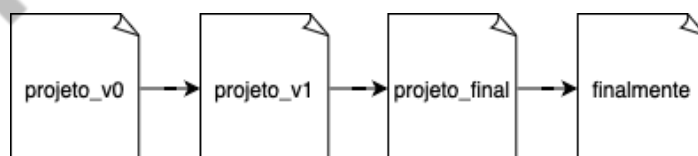


Figura 2 - Controle de versão utilizando vários arquivos
Fonte: Elaborado pelo autor (2024)

Agora, imagine que fizemos somente uma pequena alteração do arquivo projeto_v0 para o arquivo projeto_v1. Precisamos salvar todo o arquivo novo com essa pequena variação? Visando salvar espaço em disco, não seria mais interessante

salvar somente a alteração entre os arquivos? Esse processo é importante, uma vez que normalmente trabalhamos com vários arquivos. Sistemas antigos de controle de versão como é o caso do Subversion operam dessa maneira, conforme mostra a Figura 3:

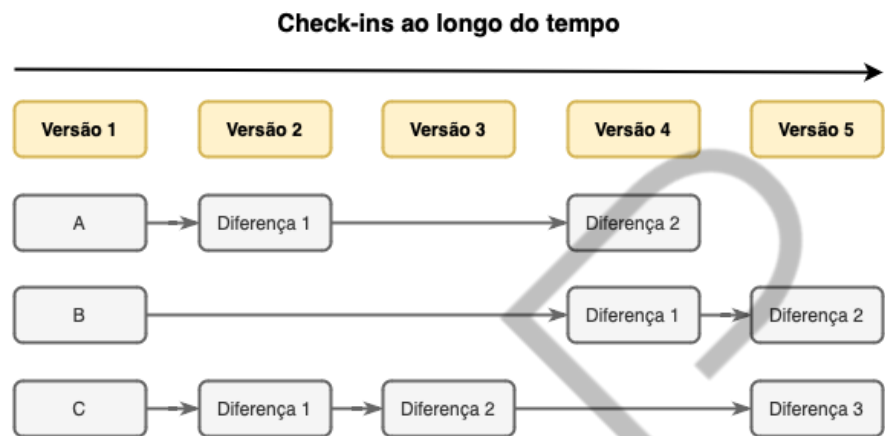


Figura 3 - Versionamento baseado em diferenças, baseado na figura do livro Pro Git
Fonte: Elaborado pelo autor (2024)

Entretanto, esse modelo não foi o que prevaleceu. A ferramenta que prevaleceu é denominada Git, a qual pensa sobre os dados como um fluxo de snapshots (cópias). As cópias, porém, só são feitas se necessário. Quando um arquivo não é modificado, se faz uma referência representada pelos retângulos com borda tracejada, conforme mostra a Figura 4:

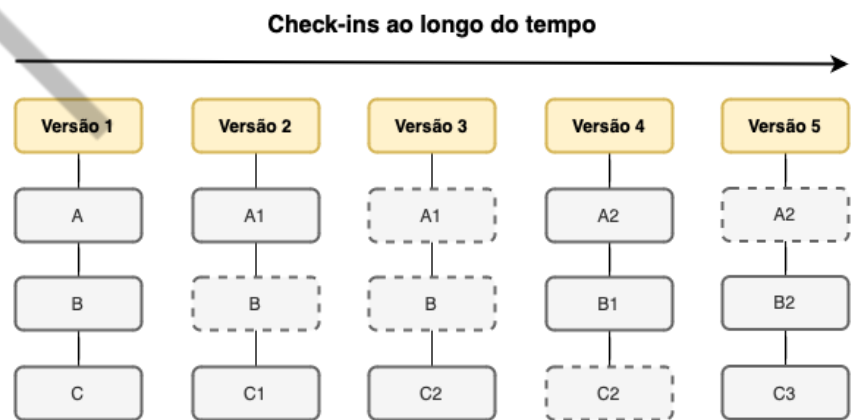


Figura 4 - Versionamento baseado em snapshots
Fonte: Elaborado pelo autor (2024)

Essa ferramenta open-source de controle de versão distribuída foi criada em 2005 para ajudar a gerenciar a construção de projetos complexos, nos quais pode existir atuação de mais de uma pessoa. Ela foi inicialmente desenvolvida por Linus Torvalds, o criador do Linux, pois durante o desenvolvimento do kernel do sistema operacional não existia uma ferramenta open-source que atendesse os requisitos de desempenho e funcionalidade para controle de versão dos códigos escritos pelas pessoas envolvidas no projeto. Inicialmente, vamos entender algumas características que o Git possui:

- A maioria das operações são locais: toda a história do seu projeto está salva no disco rígido e as operações em cima do projeto não dependem de internet (o que não é verdade para outros sistemas de controle de versão).
- Possui integridade: em todo arquivo que o git conhece é aplicado o algoritmo SHA-1, o qual gera um hash (Uma string composta por caracteres hexadecimais) baseada no conteúdo do arquivo. Logo, qualquer alteração no arquivo gera um hash diferente e o git sabe que o arquivo contém uma modificação.
- Normalmente as operações só adicionam ao banco de dados: a maioria das operações normalmente adiciona informações ao banco de dados local do Git.

Para conseguir trabalhar com o git precisamos entender os quatro estados que um arquivo possui: untracked, modified, staged e (committed / unmodified):

- Untracked: significa que esse arquivo nunca esteve no banco de dados.
- Modified: significa que o arquivo foi modificado mas ele não se encontra modificado no banco de dados.
- Staged: significa que você marcou um arquivo modificado para entrar no seu próximo commit (snapshot).
- Committed: significa que a versão do arquivo está salva no banco de dados.

Esses arquivos vão trafegar entre o seu **Diretório local**, a área denominada **Staging** e o folder **.git**. O folder **.git** contém o banco de dados que é normalmente salvo na nuvem em um serviço como Github ou Gitlab. Trazemos ele para nosso diretório local usando o comando **git clone** (também podemos usar o comando **git**

init para criar o `.git` no diretório local), depois utilizamos o comando **git add** para levar os arquivos do seu diretório local para a área de Staging. Por último, utilizamos o comando **git commit** para salvar os arquivos no banco de dados, conforme mostra a figura 5.

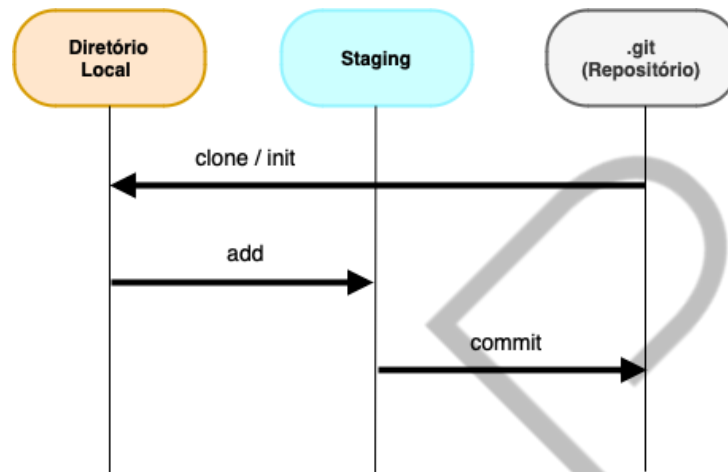


Figura 5 - Principais áreas em que os arquivos trafegam, baseado na figura do livro Pro Git
Fonte: Elaborado pelo autor (2024)

Agora vamos detalhar como o estado dos arquivos mudam entre as áreas de Diretório local e Staging, conforme mostra a Figura 6:

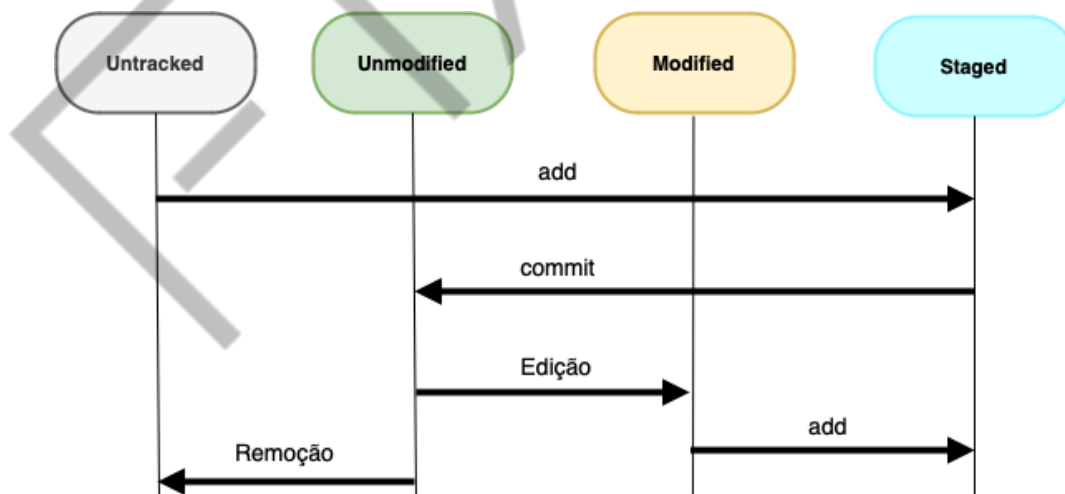


Figura 6 - Fluxo usual dos arquivos entre untracked e staging, baseado na figura do livro Pro Git
Fonte: Elaborado pelo autor (2024)

Quando criamos um arquivo novo o git não tem conhecimento sobre ele e, portanto, ele é classificado como **untracked**. Podemos aplicar nesses arquivos o

comando **git add** mudando o estado deles para **Staged**. Uma vez em staged, podemos aplicar o comando **git commit** salvando tudo na área de staging no banco de dados, mudando o estado do arquivo para **unmodified**, também denominado **committed**.

Uma vez commitado o arquivo, podemos editá-lo, e o git utilizando o algoritmo de sha-1 entende que houve modificações e muda o estado do arquivo para **modified**. Novamente, podemos salvar essas modificações aplicando o comando **git add**, seguido do comando **git commit**. Por último, podemos remover o arquivo do git utilizando o comando **git rm**, isto mudará o estado do arquivo para untracked (lembre-se de assistir a aula para ver esses comandos em ação).

Git é uma ferramenta complexa com muitos comandos e possibilidades. Antes de partir para a próxima parte, assista a videoaula e garanta que entendeu os conceitos básicos. Agora, vamos aprofundar nosso conhecimento no que são commits.

Suponha que você tem três arquivos: file1.txt, file2.txt e file3.txt. Adicione-os na área de staging e faça um commit. O git vai gerar vários objetos, conforme mostra a Figura 7. Dentre eles, vale destacar os blobs, que possuem os conteúdos dos arquivos commitados e a árvore que contém todos os arquivos referentes ao commit. Dessa forma, podemos simplificar o entendimento da ferramenta git como um simples armazenamento de dados de chave-valor (hash-árvore).

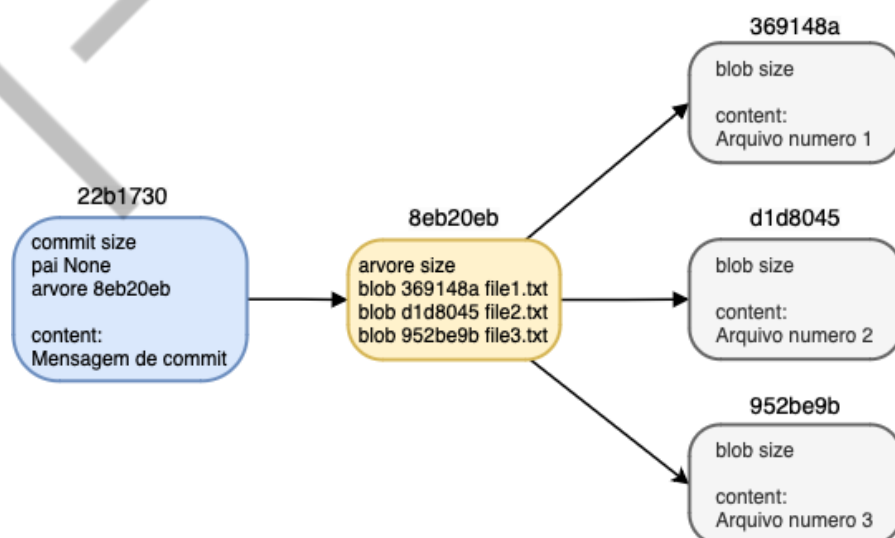


Figura 7- Estrutura de um commit, baseado na figura do livro Pro Git (2009)
Fonte: elaborado pelo autor (2024), adaptado de CHACON, STRAUB (2009)

Inicialmente, seu diretório não vai ter objetos, mas depois do commit, os objetos vão aparecer conforme a Figura 8:

```
(ins)$ tree
.
├── HEAD
├── config
├── description
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── prepare-commit-msg.sample
│   ├── push-to-checkout.sample
│   ├── sendemail-validate.sample
│   └── update.sample
├── info
│   └── exclude
├── objects
│   ├── info
│   └── pack
├── refs
│   ├── heads
│   └── tags
└──
```

```

├── COMMIT_EDITMSG
├── HEAD
├── config
├── description
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── prepare-commit-msg.sample
│   ├── push-to-checkout.sample
│   ├── sendemail-validate.sample
│   └── update.sample
├── index
├── info
│   └── exclude
├── logs
│   ├── HEAD
│   └── refs
│       ├── heads
│       └── main
├── objects
│   ├── 22
│   │   └── b1730427402ac71fe465be0ee9c042668a09a6
│   ├── 36
│   │   └── 9148afe8b19f7471ed37fe07f1e6eece159607
│   ├── 8e
│   │   └── b20ebbb47b7bf882d1bcc5ffe31caac602575f
│   ├── 95
│   │   └── 2be9b62a9ec273cc20d3a875666751effb472d
│   ├── d1
│   │   └── d8045b0c75442d90cb8c4825a00bb7fd4efe1c
│   ├── info
│   └── pack
├── refs
│   ├── heads
│   └── main
└── tags
```

Figura 8 - Exemplo de Folder .git antes (imagem a esquerda) e após (imagem a direita) um commit

Fonte: elaborado pelo autor (2024)

Perceba que os dois primeiros caracteres do hash do commit é o folder. Por último, você pode recuperar o conteúdo de qualquer hash (não precisa do hash inteiro do objeto) utilizando o comando: **git cat-file -p <hash>**, conforme Figura 9:

```
(ins)$ git cat-file -p 8eb20eb
100644 blob 369148afe8b19f7471ed37fe07f1e6eece159607    file1.txt
100644 blob d1d8045b0c75442d90cb6c4825a00bb7fd4efe1c    file2.txt
100644 blob 952be9b62a9ec273cc20d3a875666751effb472d    file3.txt
```

Figura 9 - Exemplo de um objeto gerado pelo git
Fonte: Elaborado pelo autor (2024)

Agora, suponha que façamos uma alteração no arquivo **file1.txt** e commitamos essa alteração no banco de dados. Esse commit vai ter um ponteiro indicando para o commit anterior, criando um histórico de alterações. Por último, existe um ponteiro denominado HEAD, que direciona para o commit de interesse (vamos aprofundar depois), conforme mostra a Figura 10.

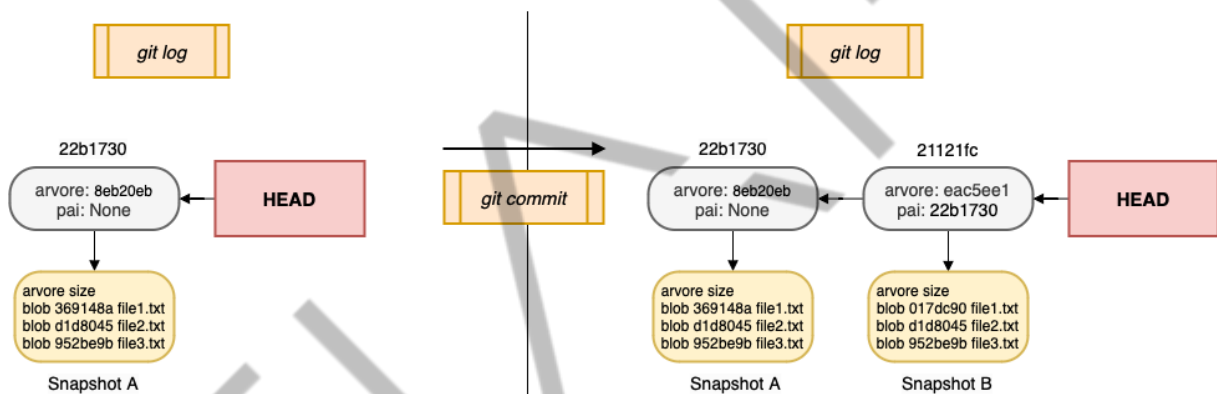


Figura 10 - Histórico de commits do git
Fonte: Elaborado pelo autor (2024)

Perceba que o commit é uma foto (snapshot) do seu projeto. Além disso, somente o **file1.txt** tem um novo blob (hash) no snapshot B, pois o git entende que não houve alterações dos outros arquivos, e logo não precisamos duplicar esses dados criando novos blobs. Para ver esse histórico de commits, podemos utilizar o comando: **git log**.

Importante notar que, toda vez que um arquivo é alterado e commitado, é gerado um novo blob para ele. Sendo assim, não é recomendado salvar um conjunto de dados ou modelos no git, pois seu projeto pode ficar muito pesado, e isso dificultará algumas operações, como por exemplo o download do repositório de um servidor remoto.

Para finalizar a nossa abordagem sobre esta ferramenta, vamos explicar sobre os temas Branching e Merging. Na discussão anterior foi omitido, mas todos commits são feitos dentro de uma **branch**.

Ela é um ponteiro móvel para um commit, e normalmente é denominada master ou main, a depender da configuração da ferramenta git, conforme mostra a Figura 11:

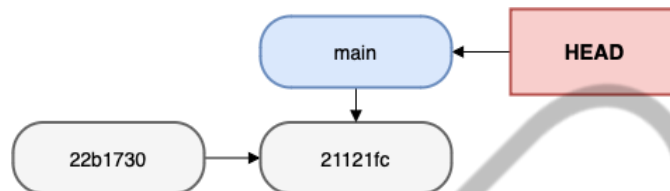


Figura 11 - Representação de uma branch em um histórico de commits, baseado na figura do livro Pro Git (2009)

Fonte: Elaborado pelo autor (2024), adaptado de CHACON, STRAUB (2009)

Podemos criar e mudar o HEAD para essa branch utilizando o comando: **git switch -c <nome-branch>** (ou usando a versão antiga: **git checkout -b <nome-branch>**). Não existe limite para o número de branches que podemos criar. Portanto, quando vamos começar um trabalho novo no repositório, normalmente criamos uma branch e fazemos os commits nessa branch, conforme mostra a Figura 12:

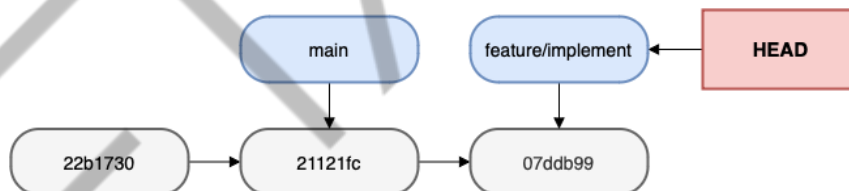


Figura 12 - Criando uma nova branch, baseado na figura do livro Pro Git

Fonte: Elaborado pelo autor (2024)

Entretanto, imagine que durante o desenvolvimento do seu código na branch **feature/implement** você percebe que o código da branch **main** contém um erro que precisa de correção urgente. Você então, usa o comando **git switch** para mudar para branch **main**, seguido do comando **git switch -c** para criar e mudar o HEAD para a branch chamada **hotfix**, conforme mostra a figura 13:

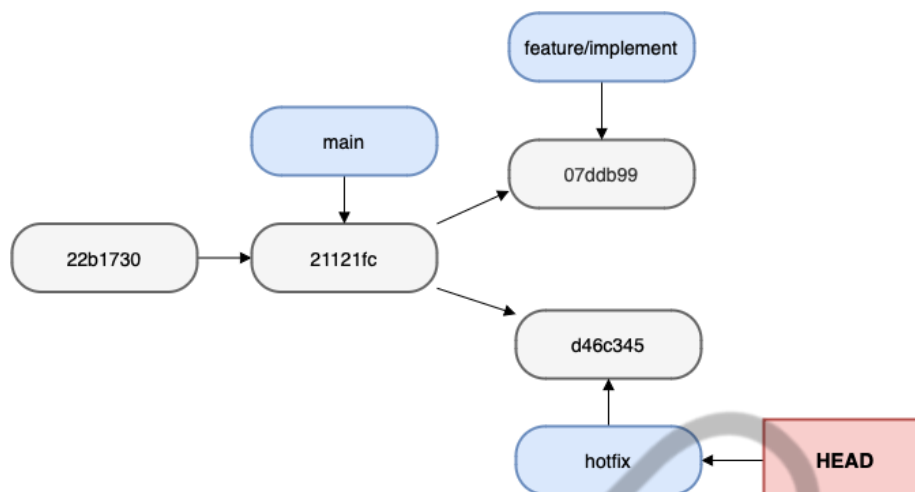


Figura 13 - Criando uma branch hotfix, baseado na figura do livro Pro Git
Fonte: Elaborado pelo autor (2024)

Depois de implementar a solução do problema, precisamos colocá-la no fluxo principal de trabalho, isto é, a branch **main**. Para fazer isso, basta voltar para branch **main** (comando **git switch**) e aplicar o comando **git merge hotfix**, conforme mostra a Figura 14:

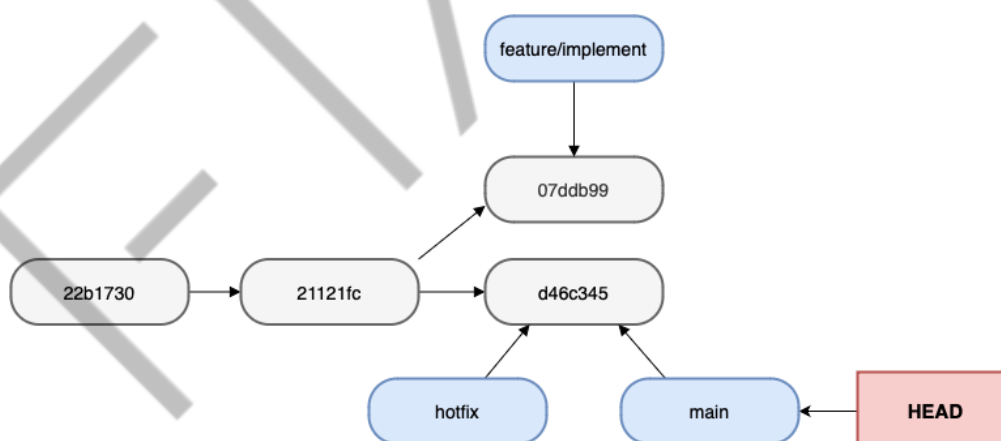


Figura 14 - Merge da branch hotfix na branch main, baseado na figura do livro Pro Git
Fonte: Elaborado pelo autor (2024)

Uma vez finalizada a correção na branch **hotfix**, a mesma pode ser deletada utilizando o comando **git branch -d hotfix**. Agora, você pode mudar para a branch **feature/implement** e continuar a implementação do trabalho, conforme a Figura 15:

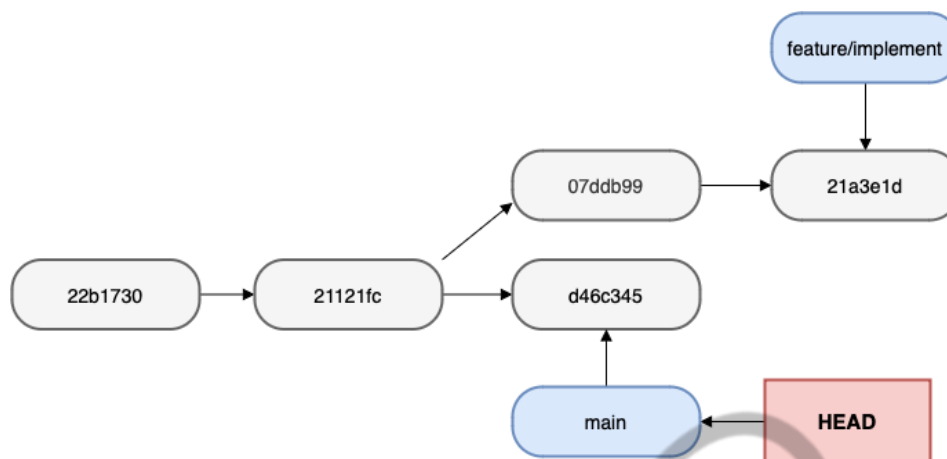


Figura 15 - Fluxo do histórico após deleção da branch e continuação da implementação na branch **feature/implement**, baseado na figura do livro Pro Git (2009)
Fonte: Elaborado pelo autor (2024), adaptado de CHACON, STRAUB (2009)

Entretanto, uma vez que a implementação esteja finalizada, queremos colocar nossa feature no fluxo de trabalho principal (branch **main**). Portanto, precisamos mudar para a branch **main** e aplicar o comando **git merge feature/implement**, conforme mostra a Figura 16:

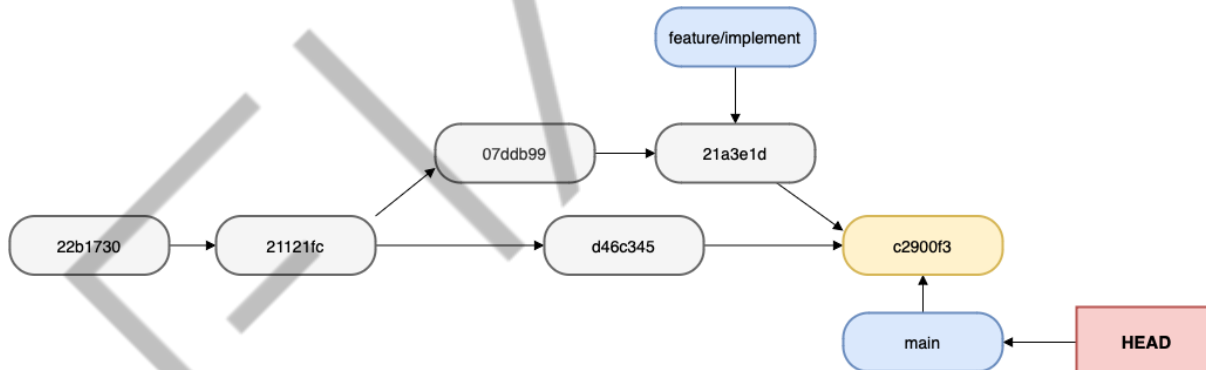


Figura 16 - Operação de merge, baseado na figura do livro Pro Git (2009)
Fonte: Elaborado pelo autor (2024), adaptado de CHACON, STRAUB (2009)

É importante notar que a operação de merge vai gerar um commit novo, representado na Figura 16 pela cor amarela. Além disso, a depender das mudanças feitas nos arquivos das branches, pode ser necessário uma intervenção manual, os quais são realizados no commit em amarelo.

O Git é uma ferramenta com muitos comandos, e algumas pessoas utilizam interfaces gráficas que abstraem esses comandos. Entretanto, é importante entender os fundamentos da ferramenta. Dependendo do fluxo de trabalho da sua equipe você

vai precisar saber sobre assuntos não explorados nesta aula, porém, sabendo os fundamentos, você vai poder continuar sua jornada de aprendizado de uma forma mais tranquila. Para finalizar, vamos descrever os comandos mais interessantes do Git para ajudar você ao longo da sua jornada.

Relacionados com repositório remotos:

- git clone: traz um repositório remoto para o computador local.
- git pull: puxa modificações remotas feitas em uma branch remota e aplica git merge.
- git push: envia as modificações locais para um repositório remoto.

Relacionados com o fluxo de salvar arquivos no banco de dados:

- git add: utilizado para adicionar arquivos na área de staging.
- git commit: utilizado para salvar um snapshot da área de stage no banco de dados local.
- git restore: utilizado para desfazer operações entre as áreas de stage e o diretório local.

Relacionados com branches:

- git switch: utilizado para alterar entre branches.
- git branch: utilizado para fazer operações em cima de branches. Por exemplo, deletar uma branch local, ou listar branches margeadas, entre outras...
- git merge: utilizado para juntar branches sem alterar o histórico de commits.
- git rebase: utilizado para juntar branches mantendo um histórico linear de commits.

Outros:

- git init: inicializa o Git em um diretório local.
- git status: utilizado para mostrar os estados dos arquivos (untracked, modified, staged e committed) nas áreas de trabalho do Git (diretório local, staging e .git).
- git diff: utilizado para mostrar diferenças entre commits e áreas de trabalho.

O QUE VOCÊ VIU NESTA AULA?

Nessa aula, você aprendeu sobre os conceitos básicos do Git. Inicialmente, aprendemos sobre as áreas de trabalho do Git, e em seguida, sobre os estados dos arquivos. Depois, aprendemos os comandos mais básicos do Git como: **git add**, **git commit** e **git status**. Para finalizar, aprendemos sobre branches e utilizamos os comandos, **git switch** e **git branch**, para criar fluxos de trabalho paralelo, seguido de como juntar esses fluxos utilizando o comando **git merge**.

REFERÊNCIAS

ENTENDENDO GIT | (não é um tutorial!). **Youtube**. 2020. Disponível em: <https://www.youtube.com/watch?v=6Czd1Yetaac&t=1291s>. Acesso em: 09 set. 2024.

STRAUB, B., CHACON, S. **Pro Git**. São Paulo: APRESS, 2009.

USANDO Git Direito | Limpando seus commits. **Youtube**. 2020. Disponível em: <https://www.youtube.com/watch?v=6OokP-NE49k>. Acesso em: 09 set. 2024.

PALAVRAS-CHAVE

Palavras-chave: Git. Controle de versão. MLOps.

EMENDAS



POSTECH